

CSP 2024 Luogu Mock Contest

SCP-J/S 2024

Junior

2024-10-13 08:30 ~ 12:00

Name	Division	Medal	Expression	Pairing
Type	Standard	Standard	Standard	Standard
Input	Standard In-put	Standard In-put	Standard In-put	Standard In-put
Output	Standard Out-put	Standard Out-put	Standard Out-put	Standard Out-put
Time Limit	1.0 second(s)	1.0 second(s)	1.0 second(s)	1.0 second(s)
Memory Limit	512 MiB	512 MiB	512 MiB	512 MiB
# of Tests	10	20	25	20
Equally Scored	yes	yes	yes	yes

Compilation Options

For C++	-O2 -std=c++14
---------	-----------------------

Division (division)

【Description】

We have learnt Euclidean division. Given two positive integers n, q , one can write down the Euclidean division equation $n \div q = k \cdots \cdots r$ or $n \div q = k(r. r)$. In this task, to simplify the conditions, even if $r = 0$, one has to write down the remainder.

Now there is an Euclidean division equation, but you only know the dividend n and the quotient k , and don't know the divisor q and the remainder r . You want to know how many possible remainders there are.

【Input Format】

Read from standard input.

Each test consists of multiple test cases. The first line contains a single integer T – the number of test cases. For each test case, there is one line containing a positive integer n and a natural number k – the dividend and the quotient.

【Output Format】

Write to the standard output.

For each test case, output a single integer – the number of possible remainders.

【Sample Input 1】

```
1 2
2 10 2
3 1 0
```

【Sample Output 1】

```
1 2
2 1
```

【Sample Explanation 1】

For the first test case, the dividend is 10 and the quotient is 2.

- If the divisor is 1, 2, 3, then the quotient is $k = 10, 5, 3$, respectively, which doesn't satisfy the conditions.

- If the divisor is 4, then the quotient is 2 and the remainder is $r = 2$.
- If the divisor is 5, then the quotient is 2 and the remainder is $r = 0$.
- If the divisor is 6, 7, 8, 9, 10, then the quotient is 1, which doesn't satisfy the conditions.
- If the divisor is greater than 10, then the quotient is 0, which doesn't satisfy the conditions.

For the second test case, the dividend is 1 and the quotient is 0.

As long as the divisor $q > 1$, $1 \div q = 0 \dots\dots 1$ is always a correct Euclidean division equation. The only possible remainder is 1.

【Sample 2】

Refer to ***division/division2.in*** and ***division/division2.ans*** in the attachments.

【Constraints】

For the first 30% of tests, $1 \leq n \leq 1000$, $0 \leq k \leq 1000$.

For the other 20% of tests, $k \leq 10^5$.

For the other 20% of tests, $k \geq 10^9$.

For all tests, $1 \leq T \leq 10$, $1 \leq n \leq 10^{14}$, $0 \leq k \leq 10^{14}$.

Medal (medal)

【Description】

There are n children who participated in several contests. The i -th child won g_i gold medals, s_i silver medals and b_i bronze medals. The teacher wants **every child** to build a ranking list of **all children**.

Every child wants themselves to rank better, so they will do some tricks on the standard of ranking. Each child can choose to sort by the number of gold medals, silver medals or bronze medals. In the ranking list, if the author ranks the same as someone else, the author can always put themselves in the front.

Given the number of gold medals, silver medals and bronze medals each child won. For each child i , calculate the best ranking in his own ranking list.

【Input Format】

Read from standard input.

The first line contains a positive integer n — the number of children.

The following n lines contains three natural numbers g_i, s_i, b_i — the number of gold medals, silver medals and bronze medals won by i -th child.

【Output Format】

Write to the standard output.

Output n lines, each line containing one positive integer. The integer on i -th line indicates the best ranking of child i .

【Sample Input 1】

```
1 4
2 8 5 0
3 4 5 3
4 4 1 2
5 2 1 1
```

【Sample Output 1】

```
1 1
2 1
```

3	2
4	3

【Sample Explanation 1】

The following shows one possible scenario. The children are sorted by the column in bold.

This is the ranking list made by the first child:

Child	Gold medal	Silver medal	Bronze medal
1	8	5	0
2	4	5	3
3	4	1	2
4	2	1	1

This is the ranking list made by the second child:

Child	Gold Medal	Silver Medal	Bronze Medal
2	4	5	3
1	8	5	0
3	4	1	2
4	2	1	1

This is the ranking list made by the third child (Notice that if the child choose to sort by gold medal, the child also ranks the second.)

Child	Gold Medal	Silver Medal	Bronze Medal
2	4	5	3
3	4	1	2
4	2	1	1
1	8	5	0

This is the ranking list made by the fourth child:

Child	Gold Medal	Silver Medal	Bronze Medal
2	4	5	3
1	8	5	0

Child	Gold Medal	Silver Medal	Bronze Medal
4	2	1	1
3	4	1	2

【Sample 2】

Refer to *medal/medal2.in* and *medal/medal2.ans* in attachment.
This sample satisfies the constraints of test 8.

【Sample 3】

Refer to *medal/medal3.in* and *medal/medal3.ans* in attachment.
This sample satisfies the constraints of test 10 .

【Constraints】

For all tests, it's guaranteed that $1 \leq n \leq 2 \times 10^5$ and $0 \leq g_i, s_i, b_i \leq 10^9$.

Test	$n \leq$	Special Property
1 ~ 2	3	
3 ~ 4	100	
5 ~ 7	1000	A
8 ~ 9		
10 ~ 12	2×10^5	A
13 ~ 15		B
16 ~ 20		

- Special Property A: g_i -s are unique, s_i -s are unique and b_i -s are unique.
- Special Property B: $1 \leq g_i, s_i, b_i \leq 2 \times 10^5$.

Expression (expr)

【Description】

The conditional operator is a special operator in C++, which shares some similarities with **if** statement. It has form **Cond?E1:E2**. If **Cond** is true, then the resulting value is **E1**, otherwise **E2**.

For example, **x>5?8:6** is an expression with conditional operator (also a piecewise constant expression, see below). If $x = 7$, then the resulting value is 8, and if $x = 3$, then the resulting value is 6.

In this task, we say S is a **piecewise constant expression** if **at least one** of the conditions is satisfied:

- S is a decimal integer **a**, for example, **243**.
- There's a decimal integer **a** and two piecewise constant expressions **p,q** such that S is **x>a?p:q**.
- There's a decimal integer **a** and two piecewise constant expressions **p,q** such that S is **x<a?p:q**.

(In conditions 2,3, if we view it as an expression with conditional operator, then $x > a$ and $x < a$ is the condition, and p, q are **E1** and **E2**, respectively.)

For example, **x>154?220:x<37?16:10** is a piecewise constant expression, because **220** and **x<37?16:10** are piecewise constant expressions, and according to the condition 2, it's a piecewise constant expression.

Given a piecewise constant expression S , and it's guaranteed that all decimal integers don't exceed m .

Yummy has q queries. In every query, given a natural number x , you're asked to calculate the resulting value of S .

【Input Format】

Read from standard input.

The first line contains two positive integers m, q — the upper bound of decimal number in the expression and the number of queries.

The second line contains a piecewise constant expression S — The expression to be calculated.

The following q lines contain an integer x — the parameter of a query.

【Output Format】

Write to the standard output.

For each query, output a single line with one integer — the resulting value of S when x is given.

【Sample Input 1】

```
1 20 5
2 x>12?x<15?4:10:x<12?14:7
3 15
4 12
5 14
6 7
7 1000000
```

【Sample Output 1】

```
1 10
2 7
3 4
4 14
5 10
```

【Sample Explanation 1】

If we insert some enters and spaces into S properly, one can get:

```
1 x>12?    //If x>12
2   x<15?  //Then judge if x<15
3     4    //If so, result in 4
4     :10  //Otherwise, result in 10
5 :x<12?  //Otherwise (if x<=12), judge if x<12
6     14   //If so, result in 14
7     :7   //Otherwise, result in 7
```

One can get the output given this procedure.

【Sample 2】

Refer to *expr/expr2.in* and *expr/expr2.ans* in the attachments.

This sample satisfies the constraints of test 6.

【Sample 3】

Refer to *expr/expr3.in* and *expr/expr3.ans* in the attachments.

This sample satisfies the constraints of test 10.

【Sample 4】

Refer to *expr/expr4.in* and *expr/expr4.ans* in the attachments.

This sample satisfies the constraints of test 19 .

【Constraints】

For all tests, it's guaranteed that $0 \leq n \leq 10^5$, $1 \leq m \leq 10^5$, $1 \leq q \leq 10^5$, $1 \leq x \leq 10^9$, and S is a piecewise constant expression.

n denotes the number of conditional operator. One can estimate the length of S using n .

Test	$n \leq$	$m \leq$	$q \leq$
1	0	10^5	10
2	1		
3 ~ 5	100	9	
6 ~ 9		10^5	
10 ~ 12	10^5	9	10^5
13 ~ 16		10^5	
17 ~ 18		9	
19 ~ 25		10^5	

Pairing (pairing)

【Description】

A sequence $s_1, \dots, s_{2k}$ is **pairing**, if and only if:

- For all $1 \leq i \leq k$, $s_{2i} = s_{2i-1}$.
- For all $1 \leq i < k$, $s_{2i} \neq s_{2i+1}$.

Note that the length of a pairing sequence must be even.

For example, 3, 3, 5, 5, 2, 2 is pairing, but 2, 2, 2, 2, 5, 5 ($s_2 = s_3$ contradicts with the second condition) or 1, 2, 3, 3, 1, 1 ($s_1 \neq s_2$ contradicts with the first condition) isn't pairing.

Given a sequence $a_1, \dots, a_n$, find the maximum length of pairing subsequence.

【Input Format】

Read from standard input.

The first line consists of a positive integer n — the length of the sequence.

The second line consists of n positive integers $a_1, \dots, a_n$.

【Output Format】

Write to the standard output.

Output a single line with one natural number, indicating the maximum length of pairing subsequence. By convention, if such non-empty pairing subsequence doesn't exist, output 0.

【Sample Input 1】

```
1 8
2 1 2 2 2 2 1 2 2
```

【Sample Output 1】

```
1 4
```

【Sample Input 2】

```
1 11
2 1 1 4 1 1 2 1000 2 5 5 4
```

【Sample Output 2】

```
1 6
```

【Sample 3】

Refer to *pairing/pairing3.in* and *pairing/pairing3.ans*.
This sample satisfies the constraints of test 3.

【Sample 4】

Refer to *pairing/pairing4.in* and *pairing/pairing4.ans*.
This sample satisfies the constraints of test 12.

【Constraints】

It's guaranteed that for all tests, $2 \leq n \leq 5 \times 10^5$, $1 \leq a_i \leq 5 \times 10^5$.

Test	$n \leq$	$a_i \leq$	Special Property
1 ~ 2	18	5×10^5	
3 ~ 5	500	500	
6 ~ 7	5000	5000	
8 ~ 9		5×10^5	
10	Every number appears at most once		
11	$a_i \leq a_{i+1}$ always hold		
12 ~ 14	Every number appears at most twice		
15 ~ 20			